

1. Create an AWS Lambda function in Python that adds a new item to a DynamoDB table called 'MusicPlaylists' every time it receives a song name and artist via a test event.

ANS

- I created a DynamoDB table named:
 - MusicPlaylists
 - The table uses the following primary key:
 - I created an AWS Lambda function named:
 - musicPlaylistFunction
 - The Lambda function receives a song name and artist through a test event and stores them in DynamoDB.
 - **Python Lambda Code**
 - The item is stored in the MusicPlaylists DynamoDB table.
 - The Lambda execution role must have permission to write to the DynamoDB table, such as dynamodb:PutItem.
2. Modify your Lambda function to read all songs from the 'MusicPlaylists' DynamoDB table and return them as a JSON array.

ANS

- I modified the Lambda function to scan the MusicPlaylists table and return all stored songs as a JSON array.
 - **Update a Lambda Code**
 - The Lambda function now stores the new song when both fields are provided and then returns all songs from DynamoDB.
 - A DynamoDB Scan reads the whole table and may not be suitable for a large production table. Pagination or a more targeted Query would be preferable at scale.
3. Set up Amazon CloudWatch Logs for your Lambda function and trigger it with a test event; then, view the logs to confirm that the song name and artist are correctly logged each time the function runs.
- Hint: Use the print() statement in Python to log details, and check the CloudWatch Logs console.

ANS

- AWS Lambda automatically sends the function's standard output to **Amazon CloudWatch Logs** when the Lambda execution role has the required logging permissions.
 - I used print() statements to log the song and artist.
- ❖ Example:

```
print(f"Song: {song}")
```

```
print(f"Artist: {artist}")
```

I triggered the Lambda function using the following test event:

```
{  
    "song": "Perfect",  
    "artist": "Ed Sheeran"  
}
```

❖ AWS Console

- CloudWatch
- Logs
- Log groups
- /aws/lambda/musicPlaylistFunction

❖ Example log output:

- Song added: Perfect
 - This confirms that the Lambda function executed and correctly logged the song and artist.

4. Simulate a bug by intentionally passing an event without the 'artist' field to your Lambda function, then use CloudWatch Logs to debug and fix the error so that the function handles missing fields gracefully.

[ANS](#)

- To simulate a bug, I used a test event without the artist field:

```
{  
    "song": "Perfect"  
}
```

Instead of directly accessing:

```
artist = event["artist"]
```

❖ if not song or not artist:

```
print("Error: Missing song or artist")  
  
return {  
    "statusCode": 400,  
    "body": json.dumps({  
        "message": "Both song and artist are required"
```

```

    })
}

Final Error-Handling Code

song = event.get("song")

artist = event.get("artist")

if not song or not artist:

    print("Error: Missing song or artist")

})
}

```

- The missing artist field is now handled gracefully instead of causing a
- This makes the Lambda function more reliable when invalid input is received.

5. Use ChatGPT to generate a step-by-step checklist for monitoring and debugging a serverless workflow that integrates Lambda with DynamoDB, then apply one suggestion from the checklist to improve your Lambda function's reliability.

ANS

❖ **Step-by-Step Checklist**

1. Check Lambda execution results
Verify that the function completes successfully and check for errors.
2. Monitor CloudWatch Logs
Use print() statements to record important information such as input values, processing steps, and errors.
3. Check Lambda permissions
Make sure the Lambda execution role has the required DynamoDB permissions.
4. Check DynamoDB data
Verify that records are being created, updated, and retrieved correctly.
5. Validate input data
Check that required fields such as song and artist are present before processing the request.

6. Handle exceptions

Use try/except blocks so unexpected errors can be logged and handled properly.

7. Monitor performance

Check Lambda duration, invocation count, errors, and throttles in CloudWatch.

8. Use alarms for important failures

CloudWatch alarms can notify administrators when Lambda errors or throttling exceed an expected level.

- This improvement prevents missing or invalid input from causing an uncontrolled Lambda failure and provides useful information in CloudWatch Logs for debugging.